Projektowanie i analiza algorytmów
Sprawozdanie
Mini-projekt 2

Filip Juchnowicz

Indeks 264176

08.05.2023

Spis treści

1 Opis zadania **3**

2 Algorytmy sortowania **3**
2.1 Quick sort 3
2.2 Merge sort 3
2.3 Bucket sort 4

3 Wyniki **4**

Spis rysunków

1 Porównanie efektywności algorytmów sortowania. 5
2 Czas filtrowania struktury. 5

Spis tabel

1 Tabela wyników algorytmów filtrowania i sortowania. 4

1 Opis zadania

Dla wybranych algorytmów sortowania przeprowadź następujące testy efektywności:

1. Wczytaj n wpisów z pliku `projekt2_dane.csv`,
2. Przefiltruj wczytane wpisy pod kątem brakujących danych,
3. Dla $n \in \{10\,000, 100\,000, 500\,000, 1\,000\,000, 1\,010\,292\}$ wczytanych wpisów:
   (a) wywołaj kolejne algorytmy sortowania i zmierz czas trwania algorytmu,
   (b) oblicz średnią i medianę ocen.

2 Algorytmy sortowania

2.1 Quick sort

Algorytm `quicksort` opart jest na metodzie `divide and conquer` (dziel i zwyciężaj). Na początku wybierany jest `pivot`, element na podstawie którego elementy są dzielone na dwa podzbiory: większe i mniejsze od `pivot'u`. Proces ten jest powtarzany, aż do uzyskania podzbiorów jednoelementowych. Wybór `pivota` wpływa na złożoność obliczeniową algorytmu.

- Złożoność czasowa:
  - optymistyczna - $O(n \log(n))$,
  - pesymistyczna - $O(n^2)$.
- Złożoność pamięciowa: $O(n)$.

2.2 Merge sort

Algorytm `mergesort` także korzysta z metody D&C, dzieląc podany zestaw elementów na mniejsze, łatwiejsze do posortowania zbiory. Algorytm dzieli elementy do uzyskania zbiorów jednoelementowych. Następnie, jak wynika z nazwy, scala je oraz sortuje. Wynikiem końcowym są posortowane elementy.

- Złożoność czasowa: $O(n \log(n))$.
- Złożoność pamięciowa: $O(n)$.

2.3 Bucket sort

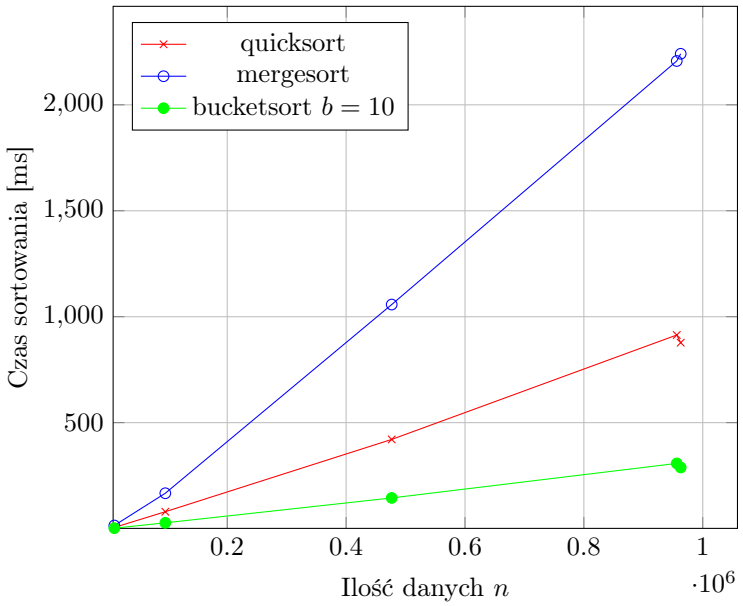
Algorytm polega na „rozrzuceniu" elementów do n kubełków (ilość kubełków i ich zakres zależy od implementacji). Następnie każdy niepusty kubełek jest sortowany, a na końcu elementy są przepisywane do oryginalnej tablicy. Istnieje szczególny przypadek kiedy ilość kubełków jest równa zakresowi danych (np. 5 kubełków dla zbioru zawierającego tylko liczby $1, 2, 3, 4, 5$). Nie wymagane jest wtedy sortowanie poszczególnych kubełków co znacząca zmniejsza czas trwania algorytmu.

- Złożoność czasowa:
  - optymistyczna - $O(n)$,
  - pesymistyczna - $O(n^2)$.
- Złożoność pamięciowa: $O(n)$.

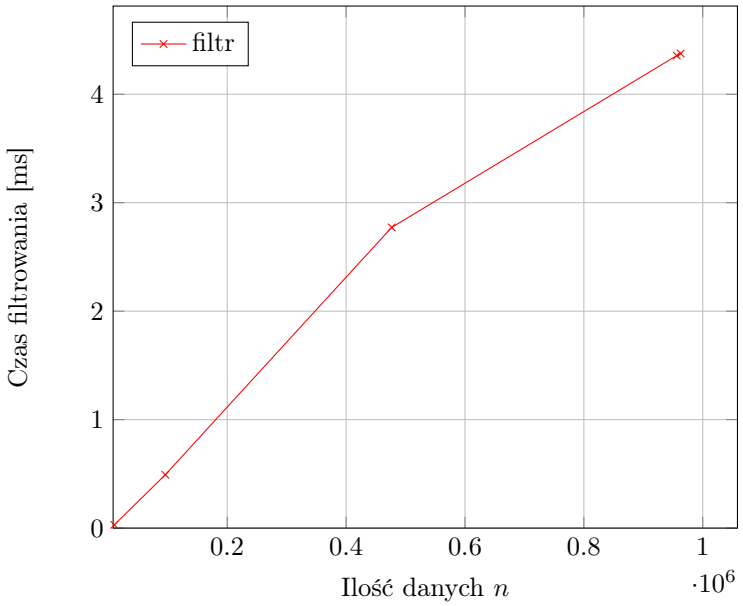
3 Wyniki

n	filtrowanie (ms)		sortowanie (ms)			
			typ	quicksort	mergesort	bucketsort
10 000	błędne	-	czas	5.293	14.623	2.069
	pozostałe	10 000	średnia	5.4603		
	czas	0.03	mediana	5		
100 000	błędne	4186	czas	79.65	166.831	27.227
	pozostałe	95 814	średnia	6.0642		
	czas	0.4922	mediana	6		
500 000	błędne	23 308	czas	420.811	1057.38	144.698
	pozostałe	476 692	średnia	6.6554		
	czas	2.7724	mediana	7		
1 000 000	błędne	43 733	czas	913.625	2206.92	307.622
	pozostałe	956 267	średnia	6.6343		
	czas	4.3564	mediana	7		
1 020 292	błędne	57 389	czas	877.419	2240.86	288.21
	pozostałe	962 903	średnia	6.6366		
	czas	4.3757	mediana	7		

Tabela 1: Tabela wyników algorytmów filtrowania i sortowania.



Rysunek 1: Porównanie efektywności algorytmów sortowania.



Rysunek 2: Czas filtrowania struktury.

Na powyższych wykresach można zauważyć, że złożoność czasowa filtrowania jest równa $O(n)$, a czasy sortowań skalują się liniowo wraz z ilością elemen-

tów. Najszybszym algorytmem jest sortowanie kubełkowe, prawdopodobnie ze względu na małą ilość możliwych wartości i odpowiednio dobraną ilością kubełków, co pozwala pominąć sortowanie wewnątrz kubełków. Najgorzej wypadł mergesort, mimo że ma podobną złożoność jak quicksort. Jest to spowodowane czasem trwania operacji, a nie ich ilością (która jest brana pod uwagę podczas szacowania złożoności czasowej).